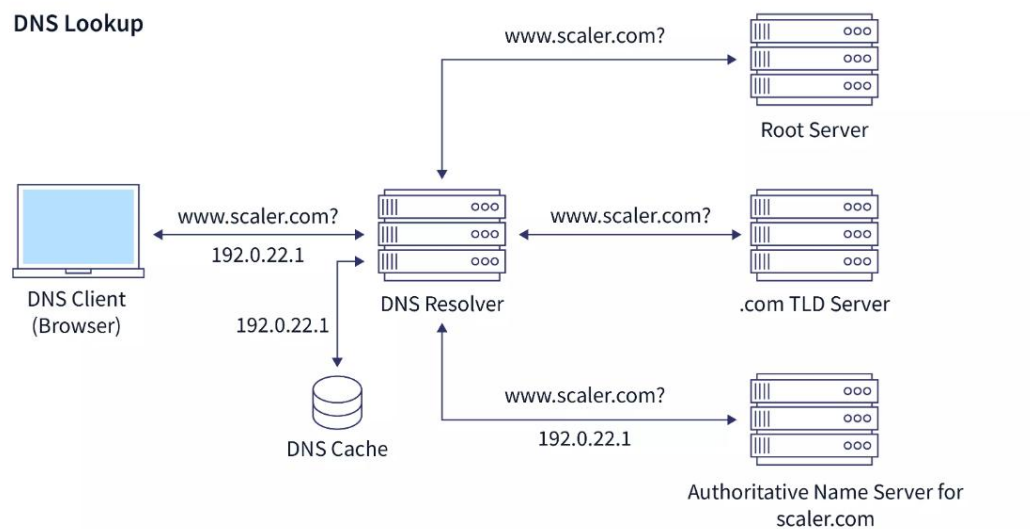


JAVA SCRIPT

- + **Dynamically typed:** Variable types are determined and checked at runtime rather than compile-time.
- + **Just-In-Time (JIT) Compiled:** convert source code into executable machine code at runtime to achieve near-native execution speeds.
- + **Single-Threaded:** JavaScript executes code sequentially on a single call stack, processing one task at a time

working of browser



client and server architecture



steps to run js:

- make js file and add in html page
- make js, install node
- open terminal and run node text.js

printing statement: `console.log();`

variables:

- think like a container (named container that stores value)

- we can store number, String, object, etc

1. var
2. let
3. const

/-----let-----

//when we declare a variable with let, we can reassign it. It is a mutable value.

//redeclaration of a variable with let is not allowed in the same scope.

```
let name="tarun";
```

```
let age=25;
```

```
console.log("My name is " + name + " and I am " + age + " years old.");
```

```
age=23; //reassign able
```

```
console.log("My name is " + name + " and I am " + age + " years old.");
```

//-----const-----

//when we declare a variable with const, we cannot reassign it. It is a constant value.

```
const pi=3.14;
```

```
console.log("The value of pi is " + pi);
```

//-----var-----

//when we declare a variable with var, we can reassign it. It is a mutable value.

//but it is not recommended to use var as it has some issues with scoping and hoisting.

```
var city="New York";
```

```
console.log("I live in " + city);
```

```
var city="Los Angeles"; //redeclaration is allowed with var
```

```
console.log("I live in " + city); //reassignable
```

-----var vs let-----

+ var is function scoped, while let is block scoped.

+ var can be redeclared and updated, while let can be updated but not redeclared in the same scope.

what is scope?

+ scope is the area of the code where a variable is defined and can be accessed.

+ There are three types of scope in JavaScript: global scope, function scope, and block scope.

Global Scope: variables declared outside of any function or block have global scope and can be accessed from anywhere in the code.

```
let b= 20; // b has global scope
```

function scope: variables declared inside a function have function scope and can only be accessed within that function.

```
function globalScopeExample() {  
  let a= 10; // a has function scope  
  console.log(a);  
  console.log(b); // b has global scope  
}  
globalScopeExample();  
//console.log(a); //a has function scope and cannot be accessed outside the function
```

Block Scope: variables declared inside a block (e.g. if statement, for loop) have block scope and can only be accessed within that block.

```
{  
  let c= 30; // c has block scope  
  console.log(c);  
  var d= 40; // d has function scope  
}  
// console.log(c); // c has block scope and cannot be accessed outside the block  
console.log(d); // d has function scope and can be accessed outside the block
```

Data Types

+ **primitives:** string, number, boolean, null, undefined, symbol, boolean, bigint

+ **non-primitives:** object, array, function

-----primitives-----

```
let str="Hello World"; //string  
console.log(typeof str); //string
```

```
let num=25; //number  
console.log(typeof num); //number
```

```
let bool=true; //boolean  
console.log(typeof bool); //boolean
```

```
let n=null; //null  
console.log(typeof n); //object
```

```
let u; //undefined  
console.log(typeof u); //undefined
```

```
let sym=Symbol("id"); //symbol
console.log(typeof sym); //symbol
```

```
let bigInt=9007199254740991n; //bigint
console.log(typeof bigInt); //bigint
```

-----non-primitives-----

```
let arr = [1,2,3,4,5];
console.log(typeof arr); //object
```

```
let obj={name:"tarun", age:25}; //object
console.log(typeof obj); //object
```

```
function greet(name) { //function
  return "Hello " + name;
}
console.log(typeof greet); //function
console.log(greet("tarun")); //function call //o/p: Hello tarun
```

operators

1. arithmetic operators -> +, -, *, /, %, **(exponential ex: 10**5 =>10^5)

```
console.log("Exponentiation: " + (a**b)); //Exponentiation: 100000
```

2. logical operators -> &&, ||, !

3. comparison operators -> ==, !=, ===, !==, <, >, <=, >= (**== only compares values but === compares values with data types**)

```
let m=10, n=20;
console.log("Equal to: " + (m==n)); //Equal to: false
console.log("Equal to: " + (m=="10")); //Equal to: true
console.log("Strictly equal to: " + (m===10)); //Strictly equal to: true
```

4. assignment operators -> =, +=, -=, *=, /=, %=

5. bitwise operators -> &, |, ^, ~, <<, >>

6. unary operators -> +, -, !, typeof, instanceof, ++, --

7. ternary operators -> condition ? expr1 : expr2

8. String Concatenation -> +

```
let fullName=firstName + " " + lastName;
console.log("Full Name: " + fullName); //Full Name: John Doe
let greeting=`Hello ${firstName} ${lastName}`; //template literal backticks important
instead of quotes in development
console.log(greeting); //Hello John Doe
```

Control flow

1. Conditional Statements: These allow the program to make decisions based on certain conditions. The most common conditional statements are if, else if, and else.
 - +They enable the execution of specific blocks of code depending on whether a condition evaluates to true or false.
 - + The switch statement evaluates an expression and executes the corresponding case block based on the value of the expression.
2. Loops: Loops are used to execute a block of code repeatedly as long as a specified condition is true. Common types of loops include for, while, and do-while loops.

Functions

1. Function declaration

```
function greet() {  
    console.log("Hello, World!");  
}  
greet();//function call
```

2. Function expression: Function can be stored in variable is called Function expression

```
Syntax: const var_name= function(){  
    .  
    function body  
    .  
}  
var_name(); // function call
```

3. Arrow function:

```
Syntax; const var_name = (Parameter) =>{ function body}  
Syntax; const var_name = (Parameter) => return_value;
```

```
const add = (a, b) => a + b;  
console.log("Sum: " + add(5, 3)); // fun call
```

4. Parameter and argument:

- + parameter: place holders in function (formal parameter)
- + arguments: actual values that are passed in function call (actual parameter)

```
function multiply(x, y) {  
    return x * y;  
}  
console.log("Product: " + multiply(4, 6));
```

5. Return Statement: to return the value

6. Default parameter: if we not pass the parameter in function call default parameter is used

```
function greetWithDefault(name = "Guest") {  
  console.log("Hello, " + name + "!");  
}  
greetWithDefault(); // Uses default parameter  
greetWithDefault("Alice"); // Overrides default parameter
```

7. Function scope:

+ variable (var, let, const) created inside function can't accessed outside fun

8. callback functions: A callback function is a just a function that you pass to another function

```
function hello(name) {  
  console.log("Hello, " + name + "!");  
}  
  
function executeCallback(callback, name) {  
  callback(name);  
}  
executeCallback(hello ,name); // syntax: callback_function( function
```

// function hello is passed as a callback to executeCallback, which then calls hello with the name "Jane Doe".

-----Object in js -----

+ An object is collection of key value pair

+ group of related data

1. creation:

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 30  
};
```

2. Access and update values

```
console.log(person.firstName);  
console.log(person.lastName); // dot notation  
console.log(person["age"]); // bracket notation
```

```
person.age = 31; // Modifying(updating) object property  
console.log(person.age);
```

3. adding or removing

```
person.address = "123 Main St"; // Adding new property
console.log(person);
```

```
delete person.address; // Deleting property
console.log(person.address); // undefined
console.log(person);
```

4. nested object

```
const student = {
  name: "Alice",
  age: 20,
  address: {
    city: "New York",
    state: "NY"
  }
};
console.log(student);
console.log(student.address.city);
```

5. object method(function inside object) and this keyword

```
const car = {
  make: "Toyota",
  model: "Camry",
  year: 2020,
  getCarInfo: function() {
    return `${this.make} ${this.model} (${this.year})`;
  }
};
```

```
console.log(car.getCarInfo());
```

```
-----
const car2 = {
  make: "Toyota",
  model: "Camry",
  year: 2020,
  getCarInfo(){
    // return `${this.make} ${this.model} (${this.year})`;/
    console.log(`${this.make} ${this.model} (${this.year})`);
  }
};
car2.getCarInfo();
```

6. looping through object:

```
for (let key in person) {  
  if (person.hasOwnProperty(key)) {  
    console.log(key + ": " + person[key]);  
  }  
}
```

Object.keys(car).forEach(key => { // Looping through object properties using Object.keys()

```
  console.log(key + ": " + car[key]);  
});
```

Object.values(person).forEach(value => { // Looping through object values using Object.values()
 console.log(value);
});

7. object destructuring: Object destructuring is a feature that lets you extract properties from an object and store them in variables in a clean, concise way.

```
const { firstName, lastName } = person; //is equivalent to const name= person.name;  
and const age = person.age;  
console.log(firstName);  
console.log(lastName);
```

prototype in js object: prototype is a built-in object from which other objects inherit properties and methods.

Syntax	Description
`const { name } = person;`	Extract a property
`const { name, age } = person;`	Extract multiple properties
`const { name: fullName } = person;`	Rename a variable
`const { age = 18 } = person;`	Set a default value
`const { name, ...rest } = person;`	Collect remaining properties
`function show({ name }) {}`	Destructure function parameters
`const { address: { city } } = person;`	Nested destructuring

-----Classes in JS-----

class is program code template for creating objects.

those objects will have some state(variables) and some behaviours(functions) inside it

```
class MyClass{
```

```
    constructor(){.....}  
    myMethod(){.....}  
}
```

let myObj=new MyClass(); // creating object

example:

```
class ToyotaCar{  
    start(){  
        console.log("start");  
    }  
  
    stop(){  
        console.log("stop");  
    }  
  
    setBrand(brand){  
        this.brand=brand;  
    }  
}
```

```
let fortuner=new fortuner();  
fortuner.setBrand("fortuner");
```

Constructor() method

- + name should be constructor
- + automatically invoked by new
- + initializes object

```
class MyClass{  
    constructor(){.....} // automatically / implicitly created  
    myMethod(){.....}  
}
```

constructor with arg:

```
constructor(arg){.....} // we can pass arguments to constructor;  
// but there is no compulsion to pass arguments if we don't pass the arguments it will be  
undefined;
```

Inheritance in JS

it is passing down properties and methods from parent class to child class.

```
class Parent {  
  ..  
}  
class Child extends Paents{  
  ..  
}
```

Note: If Child and parents have same methods, child's method will be used. (Method Overriding)

example:

```
class Parent {  
  constructor() {  
    this.name = 'Parent';  
  }  
  hello() {  
    console.log('Hello from Parent');  
  }  
  
  sleep() {  
    console.log('Parent is sleeping');  
  }  
  work() {  
    console.log('Parent is working');  
  }  
}  
  
class Child extends Parent {  
  work() {  
    super.sleep(); // Call the sleep method from Parent class  
    console.log('Child is working');  
  }  
  constructor() {  
    super();  
    this.name = 'Child';  
  }  
}
```

```
let childInstance = new Child();  
console.log(childInstance.name); // Output: Child  
childInstance.hello(); // This will call the hello method from Parent class since it's not  
overridden in Child class  
childInstance.work(); // This will call the work method from Child class
```

-----Arrays in JS-----

1. Creating of an array

2. Accessing and updating array elements

```
console.log(arr[0]);
```

```
//replace element
```

```
arr[0]=20;
```

```
console.log(arr[0]);
```

3. Array length

```
console.log(arr.length);
```

4. Iterating over an array

1. for loop

```
for(let i=0;i<colors.length;i++){
```

```
    console.log(colors[i]);
```

```
}
```

2. foreach

```
colors.forEach(a=>{
```

```
    console.log(a);
```

```
});
```

3. forof

```
for(let a of arr){
```

```
    console.log(a);
```

```
};
```

5. Array methods (push, pop, shift, unshift, splice, slice, indexOf, includes)

1. push(); // add element to end

2. pop(); // remove last

3. shift(); // remove 1st element

4. indexOf(); //get index

5. join(); //join all elements in array to String

6. slice(); // extract part

7. splice(); //removes 1 item from index 1

```
let cart=[];
```

```
//add item
```

```
cart.push("T-shirt");
```

```
cart.push("shoes");
```

```
cart.push("watch");
```

```

console.log("cart item: ",cart);

// removing item
let popItem=cart.pop();
console.log("pop item",popItem);
console.log("cart item: ",cart);

//indexOf
console.log(cart.indexOf("shoes")); // returns index of shoes

//includes
console.log(cart.includes("shoes")); // returns true if shoes is in the array

//join
console.log(cart.join(", ")); // returns string of all items in the array

//slice
let newCart=cart.slice(0,1); // returns new array with items from index 0 to 1
console.log("new cart: ",newCart);

//splice
cart.splice(1,1);// removes 1 item from index 1
console.log("cart item: ",cart);

```

6. Multidimensional arrays

```

let matrix=[
  [1,2,3],
  [4,5,6],
  [7,8,9]
];
console.log(matrix);

```

7. Array destructuring: extract values from an array and store them in variables in a simple and readable way.

```

const colors=["red", "green", "blue", "yellow", "purple", "orange"];
let [a,b,c]=colors; // first 3 values stores in a, b, c
console.log(a,b,c);

```

-----String in JS-----

```

let name="Tarun";
let message=`Welcome ${name} to the world of JavaScript`; //dynamic string

```

```
console.log(message);
```

```
name[0]="t"; // strings are immutable, this will not change the string  
console.log(name); // Output: Tarun
```

```
name="tarun"; // reassigning the variable to a new string  
console.log(name); // Output: tarun
```

```
let str="Hello World";  
console.log(str[0]); // accessing first character  
console.log(str[6]); // accessing character at index 6
```

```
console.log(str.length); // length of string
```

```
//String methods
```

```
console.log(str.length); // length of string  
console.log(str.toUpperCase()); // converts string to uppercase  
console.log(str.toLowerCase()); // converts string to lowercase  
console.log(str.indexOf("World")); // returns index of "World" in the string  
console.log(str.includes("Hello")); // returns true if "Hello" is in the string  
console.log(str.slice(0, 5)); // extracts "Hello"  
console.log(str.replace("World", "JavaScript")); // replaces "World" with "JavaScript"  
console.log(str.split(" ")); // splits the string into an array of words  
console.log(str.trim()); // removes whitespace from both ends of the string  
console.log(str.charAt(4)); // returns character at index 4
```

```
//type conversion
```

```
let num=123;  
let strNum=String(num); // converts number to string  
console.log(typeof strNum); // Output: string  
let str5=num.toString(); // another way to convert number to string
```

```
let ss="456";  
let num2=Number(ss); // converts string to number  
console.log(typeof num2);
```

```
let bool=Boolean(num2); // converts number to boolean  
console.log(bool); // Output: true
```

-----Async-await-----

async and await are JavaScript features that make asynchronous code look and behave like synchronous code, making it much easier to read and write.

async await >> promise chains >> callback hell

Sync in JS:

Synchronous:

Synchronous means the code runs in a particular sequence of instructions given in the program.

Asynchronous:

Each instruction waits for the previous instruction to complete its execution.

Due to synchronous programming, sometimes imp instructions get blocked due to some previous instructions, which causes a delay in the UI. Asynchronous code execution allows to execute next instructions immediately and doesn't block the flow.

+ Callbacks: A callback is a function passed as an argument to another function.

What is Synchronous Programming?

JavaScript executes code one line at a time.

The Problem..... Imagine downloading a large file.

```
downloadFile(); // takes 10 seconds
console.log("Done");
```

Callback Hell

Callback Hell: Nested callbacks stacked below one another forming a pyramind structure.

Promises:

+ Promise is for “eventual” completion of task. It is an object in JS.

+ It is a solution to callback hell.

```
let promise = new Promise( (resolve, reject) => { .... } )
```

Function with 2 handlers

*resolve & reject are callbacks provided by JS

Promises

A JavaScript Promise object can be:

+ Pending : the result is undefined

+ Resolved : the result is a value (fulfilled)

+ Rejected : the result is an error object

resolve(result)

reject(error)

*Promise has state (pending, fulfilled) & some result (result for resolve & error for reject).

Promises can be used by

`.then( ) & .catch( )`

`promise.then( ( res ) => { .... } )`

`promise.catch( ( err ) ) => { .... } )`

Pending



Fulfilled

or

Rejected

```
const promise = new Promise((resolve, reject) => {  
  resolve("Success");  
});
```

```
promise  
.then(result => {  
  console.log(result);  
})  
.catch(error => {  
  console.log(error);  
});
```

Async-Await:

async function always returns a promise.

async function myFunc() { }

await pauses the execution of its surrounding async function until the promise is settled.

.

The Problem..... Imagine downloading a large file.

`downloadFile(); // takes 10 seconds`

`console.log("Done");`

"Start downloading. I'll continue doing other work. Tell me when it's finished."

Start Download



Continue Running Code



Download Completes



Run Callback

ex:

```
console.log("Start");
setTimeout(() => {
  console.log("Downloaded");
}, 3000);
console.log("End");
```

output :

Start

End

Downloaded

- + JavaScript is single-threaded.
- + async always returns a Promise.
- + await waits for a Promise inside an async function.
- + await does not block the entire program—only the current async function.

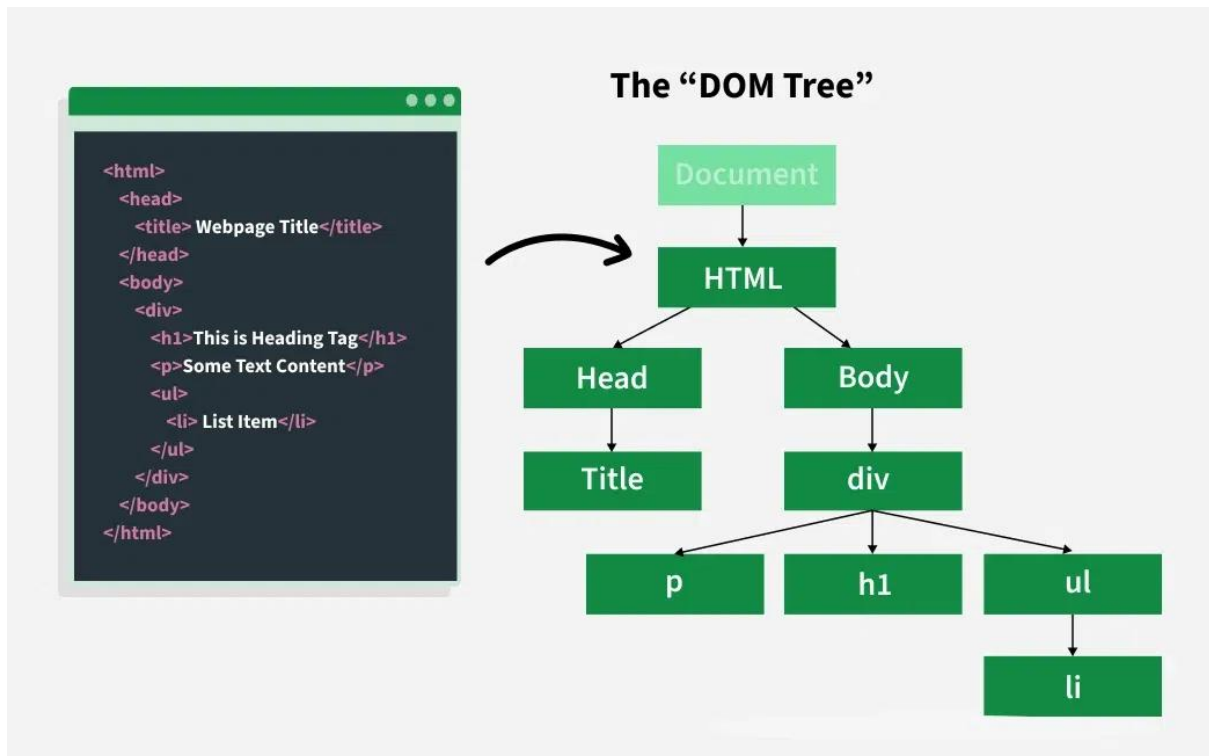
----- DOM -----

JavaScript DOM (Document Object Model) – Complete Notes

The Document Object Model (DOM) is a programming interface for HTML and XML documents. It represents the webpage as a tree of objects, allowing JavaScript to access, modify, add, or remove HTML elements dynamically.

What is the DOM?

When a browser loads an HTML document, it converts it into a tree-like structure called the DOM Tree.



- + The document object is created by the browser (Chrome, Edge, Firefox), not by Node.js.
- + Every HTML element becomes an object that JavaScript can manipulate.

Document object model

1. Accessing elements by ID:

```

const titleElement = document.getElementById("title");
console.log(titleElement.textContent); // Output: DOM Understanding
console.log(titleElement.innerHTML); // Output: DOM Understanding
console.log(titleElement.innerText); // Output: DOM Understanding
console.log(titleElement.outerHTML); // Output: <h1 id="title">DOM
Understanding</h1>
  
```

Common DOM Selection Methods

+ ----- +	
Method	Returns
-----	-----
`getElementById()`	Single element
`getElementsByClassName()`	HTMLCollection
`getElementsByTagName()`	HTMLCollection
`querySelector()`	First matching element
`querySelectorAll()`	NodeList
+ ----- +	

Accessing elements by class name

```

const descriptionElements =
  document.getElementsByClassName("description");
  
```

```
console.log(descriptionElements[0].textContent); // Output: This is a paragraph with a class attribute.
```

Accessing elements by tag name

```
const listItems = document.getElementsByTagName("li");  
console.log(listItems.length); // Output: 3  
console.log(listItems[0].textContent); // Output: Item 1
```

Accessing elements using querySelector

```
const firstListItem = document.querySelector("li");  
console.log(firstListItem.textContent); // Output: Item 1
```

accessing elements using querySelector by ID

```
const listElement = document.querySelector("#list");  
console.log(listElement.textContent); // Output: Item 1Item 2Item 3
```

accessing elements using querySelector by class

```
const descriptionElement = document.querySelector(".description");  
console.log(descriptionElement.textContent); // Output: This is a paragraph with a class attribute.
```

Accessing elements using querySelectorAll

```
const allListItems = document.querySelectorAll("li"); //Returns a NodeList of all matching elements.  
console.log(allListItems.length); // Output: 3
```

`textContent` vs `innerHTML` vs `innerText`

These three DOM properties are used to read or modify the content of an HTML element, but each behaves differently.

1. `textContent`: textContent returns or sets all the text inside an element, including text inside child elements. It ignores HTML tags and includes hidden text.

Example

```
```html  
<p id="demo">
 Hello World
 Hidden
</p>
```javascript  
const demo = document.getElementById("demo");
```

```
console.log(demo.textContent);  
...
```

Output

Hello World Hidden

Characteristics

- * Returns only text (no HTML tags).
- * Includes hidden text (`display: none`, `visibility: hidden`).
- * Ignores CSS styling.
- * Faster than `innerText` because it does not calculate styles.
- * Can also be used to replace all text inside an element.

2. `innerText`: `innerText` returns or sets **only the text that is visible on the webpage**.

Example

```
```html  
<p id="demo">
 Hello World
 Hidden
</p>
```javascript  
console.log(demo.innerText);  
...
```

Output

Hello World //The word **Hidden** is not returned because it is not visible.

Characteristics

- * Returns visible text only.
- * Ignores hidden elements.
- * Respects CSS (`display`, `visibility`, etc.).
- * Preserves line breaks as shown on the page.
- * Slightly slower because the browser must calculate the rendered layout.

3. `innerHTML`: `innerHTML` returns or sets the **HTML markup inside an element**, including all child tags.

Example

```
```html
<p id="demo">
 Hello World
</p>
```javascript
console.log(demo.innerHTML);
```
```

### ### Output

```
```html
Hello <b>World</b>
```
```

### ### Characteristics

- \* Returns HTML along with text.
- \* Includes all child elements and tags.
- \* Can be used to create or replace HTML dynamically.
- \* Useful for adding formatted content.
- \* Be careful when inserting user input, as it can lead to **Cross-Site Scripting (XSS)** vulnerabilities.

### # Visual Example

### ### HTML

```
```html
<p id="demo">
  Hello
  <b>World</b>
  <span style="display:none">Hidden</span>
</p>
```
```

### ### JavaScript

```
```javascript
const demo = document.getElementById("demo");

console.log(demo.textContent);
console.log(demo.innerText);
console.log(demo.innerHTML);
```
```

### ### Output:

|               |                                                               |  |
|---------------|---------------------------------------------------------------|--|
| `textContent` | `Hello World Hidden`                                          |  |
| `innerText`   | `Hello World`                                                 |  |
| `innerHTML`   | `Hello <b>World</b> <span style="display:none">Hidden</span>` |  |

| Feature                   | textContent                                                                           | `innerText`                                                                           | `innerHTML`                                                                                     |  |
|---------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|--|
| -----                     | -----                                                                                 | -----                                                                                 | -----                                                                                           |  |
| Returns text              |  Yes |  Yes |  Yes           |  |
| Returns HTML tags         |  No  |  No  |  Yes           |  |
| Includes hidden text      |  Yes |  No  |  Yes (as HTML) |  |
| Respects CSS              |  No  |  Yes |  No            |  |
| Preserves HTML formatting |  No  |  No  |  Yes           |  |
| Can modify HTML           |  No  |  No  |  Yes           |  |
| Performance               | Fastest                                                                               | Slower                                                                                | Moderate                                                                                        |  |

### # Quick Revision

- \* `textContent` → Plain text, includes hidden text, ignores HTML tags.
- \* `innerText` → Only visible text, respects CSS, ignores hidden content.
- \* `innerHTML` → Returns or sets HTML markup, including child elements and tags.

### -----DOM manipulation -----

```
//accessing the text content of the div
node.textContent()
```

ex:

```
const mylist=document.querySelector("#mylist");
console.log(mylist);
console.log(mylist.textContent);
console.log(mylist.innerHTML);
```

```
//modifying the HTML content of the div
node.textContent()="new Text"
```

```
mylist.innerText="This is a new text content.";
mydiv.innerHTML="<p>This is a new paragraph.</p>";
```

```
//accessing the attribute value of any node
node.getAttribute("Attribute")
```

```
//to set attribute
node.setAttribute("Attribute", "value");
```

```
//style we can access and modifies
node.style
node.style.property="value"
```

```
// Inserting Element
2 steps:
```

-----

1.Create Element

```
let el=document.createElement("node"); node-> p,div,h1-h6,li,button...etc
```

2.append(4 ways/ positions)

```
node.append(el); //adds at the end of node(inside)
node.preappend(el); //adds at the start of node(inside)
node.before(el); //adds before the node(outside)
node.after(el); //adds after the node(outside)
```

```
//Delete Element
node.remove
```

-----

-----event Handling-----

The change in the state of an object is known as an Event

- + Mouse events (click, double click etc.)
- + Keyboard events (keypress, keyup, keydown)
- + Form events (submit etc.)
- + Print event & many more

Event Handling in JS (if w defined multiframe handler last handling will override all)

-----

```
node.event = () => {
//handle here
}
```

```
btn.onclick = () => {
 console.log("btn was clicked");
}
```

+ Event Object: It is a special object that has details about the event.

All event handlers have access to the Event Object's properties and methods.

```
node.event = (e) => {
 //handle here
}
```

e.target, e.type, e.clientX, e.clientY

## Event Listeners

```

+ node.addEventListener(event, callback)
 ex: bt1.addEventListener("click", (e)=>{ console.log();});
+ node.removeEventListener(event, callback)
```

## -----Fetch API-----

```
const response=await fetch(URL);
```

- + fetch(URL) sends an HTTP request to the given URL.
- + fetch() returns a Promise.
- + The Promise resolves to a Response object.
- + await pauses the async function until that Promise is resolved.
- + So response contains the server's HTTP response, not the actual JSON data yet.

```
const data=await response.json();
```

- + response.json() reads the response body and parses it as JSON.
- + response.json() also returns a Promise.
- + await waits for that Promise to resolve.
- + data contains the actual JavaScript object/array obtained from the JSON.

Note: One important point: fetch() does not reject its Promise for HTTP errors like 404 or 500. You normally check response.ok before parsing/using the data.